Either description is abstract enough to make it easily generalizable, allowing to implement a stack in a wide range of programming languages, paradigms and systems[4].

At some point, however, we will have to move to a concrete implementation and then we will need to discuss details like:

- Where are the elements stored?
  - An *array*?
  - A *linked list*?
  - A *B-tree* on disk?
- How do we keep track of the order of insertion? (connected to the previous question)
- Will the max size of the stack be known and fixed in advance?
- Can the stack contain elements of any type or all must be of the same type?
- What happens if removal is called on an empty stack? (for instance, returning `null` versus throwing an error)

And we could keep going on with questions, but hopefully you get the idea.

1.2.2 Describing a Data Structure

The crucial part of an ADT definition is to list the set operations that it allows. This is equivalent to defining an API[5], a contract with its clients.